

Automatisation de la mise en production (CI/CD)

Stage de Lorenzo Lienard, 1^{er} Année BTS informatique Lycée Wallon

Objectif

Automatiser le déploiement de ton site vitrine. Au lieu d'utiliser un logiciel FTP (comme FileZilla) pour copier manuellement tes fichiers, nous allons configurer **GitHub**

Actions. Dès que tu créeras une version officielle (un "Tag" Git), GitHub prendra tes fichiers et les enverra tout seul sur le serveur de production.

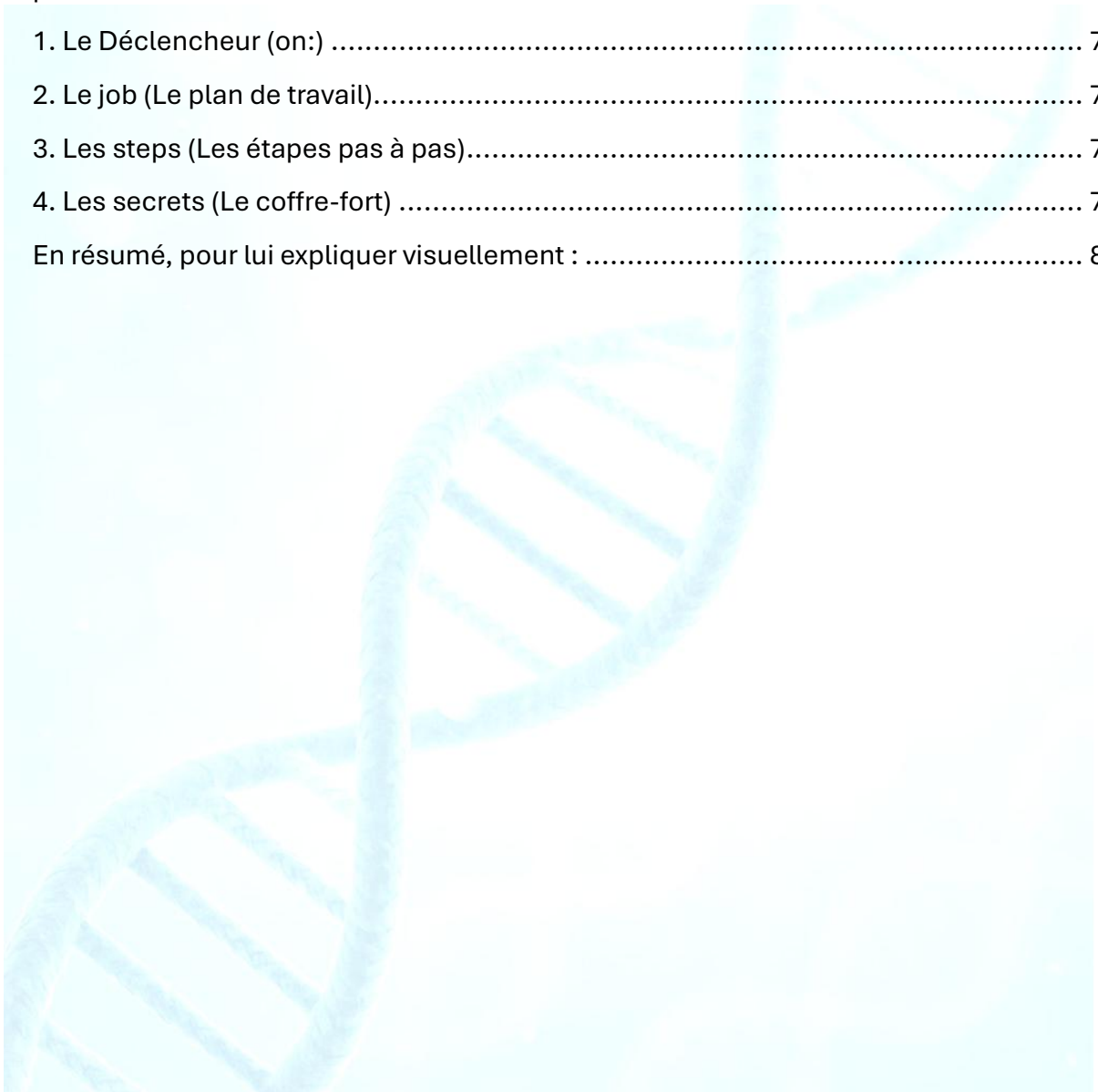
Le site sera accessible en direct à cette adresse :

<https://www.fdevelopment.eu/clinVCF/lorenzo>



Table des matières

Objectif	2
Étape 1 : Sécuriser les accès du serveur (Les "Secrets")	4
Étape 2 : Créer le script de déploiement.....	5
Étape 3 : Le test en conditions réelles (La mise en ligne).....	6
Explications:	7
1. Le Déclencheur (on:)	7
2. Le job (Le plan de travail).....	7
3. Les steps (Les étapes pas à pas).....	7
4. Les secrets (Le coffre-fort)	7
En résumé, pour lui expliquer visuellement :	8



Étape 1 : Sécuriser les accès du serveur (Les "Secrets")

Une règle d'or en développement : **On ne met jamais d'identifiants ou de mots de passe en clair dans le code.** Si ton projet GitHub est public (ou partagé), n'importe qui pourrait pirater le serveur.

Tu vas utiliser les **GitHub Secrets** pour stocker les accès FTP de manière masquée.

1. Va sur ton dépôt **GitHub** sur le web.
2. Clique sur l'onglet **Settings** (Paramètres).
3. Dans le menu de gauche, va sur **Secrets and variables > Actions**.
4. Clique sur le bouton vert **New repository secret**.
5. Ajoute les 3 secrets suivants (je te donnerai les valeurs de mon côté) :
 - **Nom** : FTP_SERVER (L'adresse de l'hébergeur)
 - **Nom** : FTP_USERNAME (L'identifiant de connexion)
 - **Nom** : FTP_PASSWORD (Le mot de passe, que je remplirai moi meme)

Étape 2 : Créer le script de déploiement

GitHub Actions lit les instructions d'automatisation dans des fichiers au format yaml placés dans un dossier très précis de ton projet.

1. À la racine de ton projet, crée un dossier nommé **.github** (attention au point).
2. À l'intérieur, crée un sous-dossier nommé workflows.
3. Dans ce dossier workflows, crée un fichier nommé deploy.yml.

Voici l'architecture visuelle simplifiée de ce que nous mettons en place : le déclenchement se fait sur l'événement du Tag, et le workflow se charge du transfert vers le disque du serveur via TCP/IP.

Ouvre le fichier deploy.yml et colle le code suivant :

```
name: Déploiement Continu FTP

# 1. On indique à GitHub QUAND déclencher l'action
on:
  push:
    tags:
      - 'v*' # Le script se déclenche uniquement si on pousse un tag qui
commence par "v" (ex: v1.0.0)

jobs:
  deploy:
    name: Envoyer les fichiers sur le serveur
    runs-on: ubuntu-latest # GitHub nous prête une machine virtuelle Linux
pour exécuter les tâches

    steps:
      # Étape A : On demande à GitHub de récupérer le code HTML/CSS de ton
dépôt
      - name: Récupération du code source
        uses: actions/checkout@v4

      # Étape B : On utilise un outil existant pour envoyer le contenu via
FTP
      - name: Synchronisation FTP
        uses: SamKirkland/FTP-Deploy-Action@v4.3.5
        with:
          server: ${ secrets.FTP_SERVER }
          username: ${ secrets.FTP_USERNAME }
          password: ${ secrets.FTP_PASSWORD }

          # Le dossier de destination sur l'hébergement

          server-dir: /clinVCF/lorenzo/
```

Ajoute ce fichier à Git (git add, git commit) et pousse-le sur ta branche principale (git push). *Note : Rien ne va se passer sur le site pour l'instant, c'est normal, car nous avons configuré le déclencheur uniquement sur les Tags.*

Étape 3 : Le test en conditions réelles (La mise en ligne)

Maintenant que le site vitrine avance et que la maquette est respectée, nous allons publier la première version officielle.

Ouvre ton terminal à la racine de ton projet et tape les commandes suivantes :

```
# 1. On crée un tag localement pour marquer la version 1.0.0
git tag v1.0.0

# 2. On envoie ce tag spécifiquement sur GitHub
git push --tags
```

Ce qu'il faut observer :

1. Va sur l'onglet **Actions** de ton projet GitHub sur le web. Tu vas voir une ligne apparaître avec un petit voyant orange qui tourne : GitHub est en train de lire ton fichier YAML et d'envoyer tes fichiers.
2. Attends que le voyant devienne **vert** (généralement moins d'une minute pour du HTML/CSS).
3. Ouvre ton navigateur et rends-toi sur :
<https://www.fdevelopment.eu/clinVCF/lorenzo>

Ton site est en ligne ! À chaque fois que tu feras des modifications importantes, il te suffira de créer un nouveau tag (ex : v1.0.1) et de faire un git push --tags pour mettre le site à jour.

Explications:

Pour un développeur qui débute, un fichier YAML de CI/CD peut ressembler à une liste de courses un peu bizarre. En réalité, c'est simplement une **recette de cuisine** bien ordonnée.

Le format **YAML** fonctionne avec un système d'indentation (les espaces/décalages). C'est ce décalage qui indique à GitHub qui dépend de quoi.

Voici l'explication des 4 concepts clés de son script, avec des mots simples :

1. Le Déclencheur (on:)

C'est le "**Quand est-ce qu'on commence ?**". Dans le script, on lui dit : « *N'agis pas à chaque fois que je modifie une ligne de code (push classique), mais attends sagement que je pousse une version officielle (un tag) qui commence par la lettre "v".* »

2. Le job (Le plan de travail)

Un **job**, c'est une grande mission globale exécutée par une machine virtuelle (un ordinateur distant) que GitHub nous prête gratuitement (ici, une machine sous Linux Ubuntu : runs-on: ubuntu-latest).

- Imagine le **job** comme un ouvrier spécialisé. Dans notre cas, l'ouvrier s'appelle deploy et sa mission est d'envoyer le site sur le serveur.

3. Les steps (Les étapes pas à pas)

Pour réussir sa mission, notre ouvrier (le *job*) doit suivre une liste d'actions dans l'ordre chronologique. Ce sont les **steps**. Si une étape échoue, tout s'arrête.

- **Étape 1 : actions/checkout@v4** La machine virtuelle de GitHub démarre complètement vide. Elle n'a pas le code du site vitrine. Cette commande magique (une action toute prête créée par GitHub) permet à la machine de "cloner" (télécharger) les fichiers HTML/CSS du dépôt pour pouvoir travailler dessus.
- **Étape 2 : SamKirkland/FTP-Deploy-Action@v4.3.5** C'est une autre action partagée par la communauté. Au lieu de réécrire tout le code pour gérer une connexion FTP, on utilise cet outil. On lui donne juste les paramètres nécessaires (l'adresse du serveur, le dossier de destination /clinVCF/lorenzo/, etc.) et il s'occupe de transférer les fichiers.

4. Les secrets (Le coffre-fort)

C'est le point le plus important pour sa culture de développeur.

La syntaxe `{{ secrets.FTP_SERVER }}` est une variable de sécurité. Elle dit à GitHub : « *Ne cherche pas ce texte dans le code, va plutôt le lire dans le coffre-fort secret du projet sur le web* ».

Pourquoi c'est crucial ? Si le stagiaire écrivait son vrai mot de passe en clair dans le fichier `deploy.yml`, ce mot de passe serait visible par toute personne ayant accès au code (et indexé par les robots si le projet devenait public). En utilisant les secrets, le mot de passe est masqué (il apparaîtra sous forme de `***` dans les lignes de log du déploiement).

En résumé, pour lui expliquer visuellement :

- **on** = L'événement déclencheur (Le coup de sifflet).
- **jobs** = Qui travaille et sur quel ordinateur ? (L'ouvrier).
- **steps** = La liste des tâches à faire dans l'ordre (La recette).
- **secrets** = Les clés du coffre-fort pour travailler en sécurité.